

|                       |            |
|-----------------------|------------|
| <b>Nome do Aluno:</b> | <b>RA:</b> |
|-----------------------|------------|

|                                                           |                                 |
|-----------------------------------------------------------|---------------------------------|
| <b>Curso:</b> Ciência da Computação                       |                                 |
| <b>Disciplina:</b> Paradigma de Linguagens de Programação | <b>Campus:</b> Passo Fundo      |
| <b>Professor (a):</b> Luciano Rodrigo Ferretto            | <b>Data:</b> ____ / ____ / 2026 |

### CARTÃO DE RESPOSTAS

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Assinatura do Aluno</p> | <p><b>ATENÇÃO! Instruções para preenchimento</b></p> <ol style="list-style-type: none"> <li>1. Não dobre ou rasure este cartão de respostas</li> <li>2. Marque apenas uma resposta por questão</li> <li>3. Todas as questões devem ser respondidas com caneta azul ou preta</li> <li>4. Nas dissertativas, caso o espaço não seja suficiente, pode-se usar o verso da folha.</li> <li>5. Preencher totalmente o espaço conforme o modelo abaixo:</li> </ol> <p style="text-align: center;"> <input checked="" type="radio"/> Forma correta de preenchimento         </p> |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Questões Objetivas

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 |
| A  | A  | A  | A  | A  | A  | A  | A  | A  | A  | A  | A  | A  | A  | A  |
| B  | B  | B  | B  | B  | B  | B  | B  | B  | B  | B  | B  | B  | B  | B  |
| C  | C  | C  | C  | C  | C  | C  | C  | C  | C  | C  | C  | C  | C  | C  |
| D  | D  | D  | D  | D  | D  | D  | D  | D  | D  | D  | D  | D  | D  | D  |
| E  | E  | E  | E  | E  | E  | E  | E  | E  | E  | E  | E  | E  | E  | E  |

#### Questões Dissertativas nas próximas folhas

Orientações:

- A prova teórica é individual e sem consulta ao material.
- **Lembre-se de colocar as respostas das questões objetivas no cartão da primeira página, e nas páginas seguintes as respostas das questões dissertativas.**
  - **NÃO SERÁ CORRIGIDO NADA ALÉM DESSAS 2 (DUAS) PRIMEIRAS PÁGINAS.**
- Questões Objetivas: 6,2
- Questões Dissertativas: 0,8

**UMA ÓTIMA PROVA A TODOS!**



## A large, faint, light gray letter 'A' is centered on a background of horizontal ruling lines. The letter is composed of two slanted strokes meeting at a point at the top, with a horizontal crossbar. The ruling lines are thin and evenly spaced, extending across the entire width of the image. The letter 'A' is positioned such that its top point is near the top of the page, and its base is near the bottom. The overall appearance is that of a piece of lined paper with a large, watermark-like letter 'A' in the center.

**1)** Durante o desenvolvimento de um sistema, um programador decide representar entidades do domínio por meio de classes que encapsulam atributos e comportamentos. Algumas classes especializadas reutilizam características de classes mais gerais, enquanto determinados métodos apresentam comportamentos diferentes de acordo com o objeto que os executa.

Considerando os conceitos da Programação Orientada a Objetos, analise as afirmações.

- I.** A utilização de classes para representar entidades está relacionada à modelagem por objetos.
- II.** A reutilização de características de uma classe por outra está relacionada à herança.
- III.** A possibilidade de um mesmo método apresentar comportamentos distintos está relacionada ao polimorfismo.
- IV.** O encapsulamento exige que todos os atributos de uma classe sejam obrigatoriamente públicos.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e III estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

---

**2)** Considere duas abordagens utilizadas para resolver um mesmo problema computacional. Na primeira, o programador descreve explicitamente as etapas que devem ser executadas para produzir o resultado. Na segunda, o programador descreve propriedades ou relações do problema, deixando parte do processo de obtenção da solução a cargo da linguagem.

A principal diferença entre essas abordagens está relacionada à distinção entre:

- A)** programação estruturada e programação orientada a objetos.
- B)** paradigma procedural e paradigma orientado a objetos.
- C)** paradigma imperativo e paradigma declarativo.
- D)** paradigma funcional e paradigma orientado a objetos.
- E)** paradigma lógico e paradigma funcional.

---

**3)** Uma equipe desenvolve uma aplicação utilizando funções que podem ser armazenadas em variáveis, passadas como argumentos para outras funções e retornadas como resultado de outras funções.

Considerando os conceitos do paradigma funcional, essa característica corresponde ao conceito de:

- A)** funções puras, relacionadas à ausência de efeitos colaterais.
- B)** funções de primeira classe, tratadas como valores no programa.
- C)** imutabilidade, relacionada à preservação do estado dos dados.
- D)** composição de funções, relacionada à combinação de transformações.
- E)** recursão, relacionada à repetição de uma operação por chamadas sucessivas.

---

**4)** Um sistema baseado em programação lógica contém os seguintes elementos:

- pai(joao, maria)
- pai(joao, pedro)
- uma regra que relaciona uma pessoa aos seus filhos;
- consultas realizadas pelo usuário.

Nesse tipo de abordagem, o programador não precisa definir necessariamente uma sequência detalhada de comandos para percorrer os dados. Em vez disso, descreve informações e relações que poderão ser utilizadas para responder às consultas.

Considerando o paradigma lógico, essa abordagem caracteriza-se principalmente por:

- A)** definir fatos e regras para que consultas possam produzir respostas por inferência.
- B)** utilizar objetos para encapsular os dados e métodos responsáveis pelas consultas.
- C)** utilizar procedimentos para controlar passo a passo o fluxo das consultas.
- D)** combinar funções puras para transformar os fatos em novos objetos de domínio.
- E)** modificar variáveis globais até que uma condição relacionada à consulta seja satisfeita.

---

**5)** Um estudante afirma que uma linguagem de programação só pode pertencer a um único paradigma. Outro estudante argumenta que uma linguagem pode oferecer recursos que permitam diferentes estilos de programação.

Considerando o conceito de linguagens multiparadigma, analise as afirmações.

- I.** Uma linguagem multiparadigma pode oferecer recursos associados a mais de um paradigma.
- II.** Diferentes partes de um mesmo sistema podem utilizar diferentes formas de estruturar suas soluções.
- III.** O uso de múltiplos paradigmas significa necessariamente que todos os paradigmas disponíveis devem ser utilizados em todos os módulos.
- IV.** A utilização de diferentes paradigmas pode ampliar as alternativas disponíveis para solucionar determinados problemas.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e IV estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

---

**6)** Durante a manutenção de um sistema, um desenvolvedor identifica cinco métodos diferentes contendo praticamente a mesma lógica para validar um CPF. Ele decide criar um único método responsável pela validação e reutilizá-lo nas diferentes partes da aplicação.

A decisão tomada está diretamente relacionada ao princípio:

- A)** KISS, pois reduz a quantidade total de métodos existentes no sistema.
- B)** YAGNI, pois elimina funcionalidades que podem não ser utilizadas.
- C)** DRY, pois evita a repetição de uma mesma lógica em diferentes locais.
- D)** SRP, pois determina que todas as validações devem ficar na mesma classe.
- E)** OCP, pois impede que o método de validação seja alterado futuramente.

---

**7)** Uma equipe está desenvolvendo um sistema de vendas. Para simplificar uma funcionalidade, um desenvolvedor propõe uma solução extremamente complexa, envolvendo várias abstrações e componentes, embora o requisito possa ser atendido por uma implementação simples.

Considerando os princípios estudados, a situação está mais diretamente relacionada ao:

- A)** YAGNI, pois o problema envolve uma funcionalidade que ainda não foi solicitada.
- B)** DRY, pois toda solução complexa necessariamente duplica algum código.
- C)** KISS, pois soluções devem evitar complexidade desnecessária.
- D)** ISP, pois interfaces complexas devem ser sempre divididas em menores.
- E)** LSP, pois componentes complexos não podem ser substituídos por componentes simples.

---

**8)** Considere as seguintes situações encontradas em um sistema:

- I.** Uma classe reúne exclusivamente métodos relacionados ao cadastro de clientes.
- II.** Uma alteração em uma classe frequentemente exige alterações em várias outras classes.
- III.** Uma classe possui métodos relacionados a clientes, geração de relatórios, envio de e-mails e processamento de pagamentos.
- IV.** Dois módulos possuem poucas dependências diretas entre si.

Considerando os conceitos de coesão e acoplamento, quais situações representam características desejáveis de projeto?

- A)** Apenas I e II.
- B)** Apenas I e IV.
- C)** Apenas II e III.
- D)** Apenas III e IV.
- E)** Apenas I, II e IV.

---

**9)** Uma classe Pedido possui um método que realiza três atividades distintas: valida os dados do pedido, calcula valores financeiros e envia notificações ao cliente. Alterações na regra de cálculo frequentemente exigem modificações no mesmo método responsável pelas notificações.

Considerando os princípios SOLID, qual abordagem está mais alinhada ao problema apresentado?

- A)** Separar as responsabilidades da classe em componentes especializados.
- B)** Criar uma única interface contendo todas as operações relacionadas ao pedido.

- C) Utilizar herança para colocar todas as responsabilidades em uma classe-base.
- D) Concentrar as regras adicionais no mesmo método para reduzir a quantidade de classes.
- E) Substituir todas as dependências por atributos públicos para facilitar as alterações.

10) Uma aplicação possui uma classe Notificacao que envia mensagens por e-mail. Posteriormente, surge a necessidade de também enviar notificações por SMS e outros meios.

O desenvolvedor altera a classe existente sempre que surge um novo tipo de notificação, adicionando condições para identificar o meio utilizado.

Considerando os princípios SOLID, analise as afirmações.

- I. A implementação dificulta a extensão da classe para novos tipos de notificação.
- II. O cenário pode ser relacionado ao Princípio Aberto/Fechado.
- III. Uma abstração para diferentes formas de notificação pode permitir a adição de novos tipos sem alterar continuamente a classe principal.
- IV. O OCP determina que uma classe nunca pode ser modificada, mesmo quando um erro existente precisa ser corrigido.

Assinale a alternativa correta.

- A) Apenas I e II estão corretas.
- B) Apenas I e III estão corretas.
- C) Apenas I, II e III estão corretas.
- D) Apenas II, III e IV estão corretas.
- E) I, II, III e IV estão corretas.

11) Uma empresa possui uma aplicação monolítica que concentra diversas funcionalidades em um único artefato. Durante a evolução do sistema, uma alteração em determinada funcionalidade exige que toda a aplicação seja construída e implantada novamente.

A equipe considera dividir a aplicação em serviços independentes.

Nesse contexto, uma característica associada à arquitetura de microservices é:

- A) concentrar todas as funcionalidades em uma única unidade de implantação.
- B) permitir que diferentes serviços sejam desenvolvidos e implantados de maneira independente.
- C) utilizar obrigatoriamente o mesmo banco de dados por todos os serviços.
- D) eliminar a necessidade de comunicação entre as diferentes partes da aplicação.
- E) garantir que todos os serviços tenham exatamente os mesmos requisitos de infraestrutura.

12) Uma organização está adotando o **Database per Service Pattern**. Durante o projeto, a equipe percebe que os serviços possuem necessidades diferentes de armazenamento: um serviço precisa realizar consultas textuais, enquanto outro necessita de transações relacionais.

Considerando o pattern estudado, qual alternativa representa uma possibilidade proporcionada por essa arquitetura?

- A) Todos os serviços precisam utilizar o mesmo tipo de banco para garantir compatibilidade.
- B) Cada serviço pode escolher uma tecnologia de persistência adequada às suas necessidades.
- C) Todos os serviços devem compartilhar as mesmas tabelas para evitar duplicação dos dados.
- D) Cada serviço deve manter uma cópia completa dos bancos de todos os demais serviços.
- E) O banco de cada serviço deve obrigatoriamente ser um banco relacional independente.

13) Uma equipe utiliza Flyway para controlar a evolução do banco de dados de um microservice. O repositório do projeto contém os seguintes arquivos:

V1\_\_criar\_tabela\_produto.sql

V2\_\_adicionar\_preco.sql

V3\_\_criar\_indice.sql

Considerando a convenção de migrações versionadas, a interpretação correta é:

- A) V1, V2 e V3 representam scripts que podem ser executados repetidamente em qualquer ordem.
- B) V1, V2 e V3 representam versões sequenciais das alterações aplicadas ao banco.
- C) V1, V2 e V3 representam diferentes ambientes de execução da aplicação.

- D) V1, V2 e V3 representam backups independentes realizados antes de alterações.  
E) V1, V2 e V3 representam arquivos utilizados apenas durante o desenvolvimento local.

14) Uma empresa possui diversos microservices. Cada serviço utiliza configurações específicas, como endereço do banco, URL de outros serviços e nível de log. Para evitar que essas informações fiquem incorporadas diretamente no código da aplicação, a equipe decide fornecê-las durante a execução.

Essa decisão está diretamente relacionada ao conceito de:

- A) Database per Service, pois cada serviço possui configurações próprias.  
B) Service Discovery, pois as configurações identificam os serviços disponíveis.  
C) Externalized Configuration, pois as configurações são fornecidas fora do código.  
D) Database Migration, pois as configurações são armazenadas em arquivos versionados.  
E) Interface Segregation, pois cada serviço recebe somente as configurações utilizadas.

15) Uma equipe utiliza um servidor centralizado para armazenar as configurações de diversos microservices. Cada aplicação consulta esse servidor para obter os valores correspondentes ao seu serviço e ao ambiente de execução.

Considerando o **Centralized Configuration Pattern**, analise as afirmações.

- I. A centralização reduz a dispersão das configurações entre diferentes projetos.  
II. O mecanismo pode facilitar a organização e a manutenção das configurações.  
III. A centralização significa que todos os serviços precisam utilizar exatamente os mesmos valores de configuração.  
IV. Recursos como atualização dinâmica dependem da solução e da implementação adotadas.

Assinale a alternativa correta.

- A) Apenas I e II estão corretas.  
B) Apenas I, II e IV estão corretas.  
C) Apenas I, III e IV estão corretas.  
D) Apenas II, III e IV estão corretas.  
E) I, II, III e IV estão corretas.

16) Considere o código Java a seguir:

```
public class ProcessadorPedido {  
  
    public void processar(Pedido pedido) {  
  
        MySQLRepositorio repositorio = new MySQLRepositorio();  
  
        repositorio.salvar(pedido);  
  
        System.out.println("Pedido processado.");  
    }  
}
```

A classe ProcessadorPedido é responsável por processar pedidos. Para armazenar o pedido, entretanto, ela cria diretamente uma instância da classe concreta MySQLRepositorio.

Posteriormente, o sistema poderá utilizar outras formas de persistência, como um repositório baseado em PostgreSQL ou outro mecanismo de armazenamento. Nesse caso, a implementação atual exigiria alterações na classe ProcessadorPedido.

**Com base nos princípios SOLID estudados na disciplina:**

- a) Identifique o princípio SOLID diretamente relacionado ao problema apresentado e explique por que a implementação atual viola esse princípio.  
b) Proponha uma solução de projeto que permita que ProcessadorPedido utilize diferentes implementações de persistência sem depender diretamente de uma classe concreta. Apresente as principais interfaces/classes envolvidas e explique como sua solução resolve o problema.